MANIPAL UNIVERSITY
JAIPUR

PROJECT REPORT

Strategic Decision-Making in Cloud Resource Allocation

A Game-Theoretic Pricing Competition Model

Submitted as part of the coursework for
Game Theory

Submitted by

Name	Registration Number
Rishi Bhardwaj	2427030397
Jaskirat Singh	2427030438
Yash Tyagi	2427030042
Kunal Tyagi	2427030660

Bachelor of Technology
Computer Science and Engineering
Semester V

Manipal University Jaipur

October 2026

Abstract

This project examines strategic pricing among competing cloud-resource providers. Each provider selects a price to maximise profit, while customer demand is distributed using a multinomial logit choice model. The allocation depends on relative prices and a price-sensitivity parameter. An iterative best-response procedure searches a finite set of candidate prices to obtain a numerical approximation to a pure-strategy Nash equilibrium.

The implementation consists of a Python model using NumPy and Matplotlib and an interactive browser dashboard, Cloud Pricing Lab. Two experiments are considered: a three-provider market with different marginal costs and a comparison of markets containing two to six providers with equal marginal costs. In the three-provider example, equilibrium prices are approximately 3.126, 3.364 and 3.269 model units. The lowest-cost provider receives the largest demand allocation and earns the highest profit. In the symmetric comparison, average equilibrium price decreases as the number of providers increases.

The results illustrate the stated model rather than actual cloud-market prices. Demand is fixed within each scenario, all customers are allocated among the modeled providers, and capacity constraints are not enforced. These assumptions define the scope of the conclusions.

Keywords: game theory; cloud pricing; Nash equilibrium; best-response dynamics; multinomial logit.

Contents

1. Introduction and Objectives 3

2. Game Formulation 4

3. Methodology and Implementation 5

4. Results: Three-Provider Market 6

5. Results: Competition Experiment 7

6. Discussion, Limitations and Conclusion 8

References and Reproducibility 9

Appendix A. Dashboard Screenshots 10-11

1. Introduction and Objectives

1.1 Background

Cloud-computing providers sell access to resources such as processing power. When competing services are comparable, the price charged by one provider influences how much demand it receives relative to its competitors. A higher price increases the margin on each unit sold but can reduce the provider's share of customers. A lower price can attract demand while reducing the margin.

This interdependence makes pricing a strategic decision. A provider must consider both its own cost and the prices selected by its rivals. Game theory provides a framework for describing these decisions and identifying outcomes in which no provider benefits from changing its price alone [1, 3].

1.2 Problem Statement

The project investigates how customer demand is allocated among competing cloud providers and which prices remain stable under individual profit maximisation. In this report, resource allocation refers to the distribution of modeled customer demand among providers. It does not refer to scheduling workloads on physical servers or deploying cloud infrastructure.

1.3 Objectives

1. Represent providers as strategic players whose decision variable is price.

2. Allocate customer demand using a price-sensitive multinomial logit model.

3. Calculate provider profit from price, marginal cost and allocated demand.

4. Approximate a pure-strategy Nash equilibrium using iterative best responses.

5. Compare providers with different costs and markets with two to six competitors.

6. Provide an interactive dashboard for exploring the model assumptions.

1.4 Scope

AWS_like, Azure_like and GCP_like are illustrative labels. Costs and demand are not calibrated to real providers. Prices and profits use abstract model units; no currency, workload specification or billing period is assigned. The application has no connection to a cloud-provider API.

2. Game Formulation

2.1 Players, Strategies and Payoffs

The providers are the players in a non-cooperative pricing game. Each provider chooses one price from a grid of 200 values between 0.5 and 10.0. Its payoff is its profit. A best response is the candidate price giving the highest profit when all competitors' prices are held fixed. A Nash equilibrium is a price vector at which no provider can increase its payoff through a unilateral change [1, 3].

2.2 Demand Allocation

Let p_i be provider i's price, c_i its marginal cost, D_0 total market demand, and α the price-sensitivity parameter. The model uses the following price-based utility and logit share [2, 4]:

$$U_i = -\alpha p_i$$

$$s_i = \exp(-\alpha p_i) / \Sigma_j \exp(-\alpha p_j)$$

$$D_i = D_0 s_i$$

A lower price produces a larger share, other prices being unchanged. A higher value of α makes the shares more sensitive to price differences. The shares sum to one, so the entire configured demand is allocated among the active providers. Customers have no outside option to select an unmodeled provider or make no purchase.

2.3 Profit Function

$$\pi_i = (p_i - c_i)D_i$$

The term $p_i - c_i$ is the margin per unit. A provider therefore balances margin against the demand received. Fixed costs are excluded from this profit calculation.

2.4 Equilibrium Interpretation

The reported solution is a grid-based numerical approximation to a pure-strategy Nash equilibrium. It identifies a fixed price for each provider rather than a probability distribution over prices. It does not imply equal profits, maximum industry profit or the lowest possible consumer prices. The finite price range and grid resolution limit the precision of the solution.

3. Methodology and Implementation

3.1 Best-Response Procedure

1. Initialise each provider's price at 1.5 times its marginal cost.

2. For each provider, evaluate 200 evenly spaced candidate prices in the interval [0.5, 10.0], using the previous round's prices for all competitors.

3. Select the profit-maximising candidate for each provider and apply the new prices simultaneously.

4. Repeat until the largest absolute price change is below 0.001, or until 100 iterations have been completed.

A stable vector is interpreted as a grid-based equilibrium. Reaching the iteration limit alone does not establish equilibrium; the convergence status must also be considered.

3.2 Experimental Settings

Parameter	Three-provider market	Competition comparison
Provider count	3	2, 3, 4, 5 and 6
Marginal costs	1.8, 2.2 and 2.0	2.0 for every provider
Base demand	2,000	1,000 × N / 3
Price sensitivity α	1.2	1.5
Price grid	0.5 to 10.0; 200 points	0.5 to 10.0; 200 points

3.3 Software Structure

The Python file cloud_pricing_game.py contains a CloudProvider data class and a CloudPricingGame class. NumPy is used for arrays, exponentials and numerical calculations; Matplotlib produces the figures. The model implements demand, profit, discrete best-response search, equilibrium iteration and the competition comparison. An optional capacity field is defined but is not enforced in the calculations.

3.4 Interactive Dashboard

Cloud Pricing Lab implements the same model in browser JavaScript. Users can set demand from 500 to 5,000 units, adjust price sensitivity, and edit the names and marginal costs of two to six providers. The dashboard displays the equilibrium table, price convergence, demand and profit, and a separate competition experiment. It runs entirely in the browser; Python is used independently for the original analysis and plots.

4. Results: Three-Provider Market

The first experiment uses marginal costs of 1.8, 2.2 and 2.0, total demand of 2,000 units and α = 1.2. Table 1 reports the equilibrium output. Values are rounded for presentation and remain in abstract model units.

Table 1. Equilibrium outcomes for providers with different marginal costs.

Provider	Cost	Price	Demand	Profit
AWS_like	1.80	3.126	771.3	1,022.5
Azure_like	2.20	3.364	579.2	674.4
GCP_like	2.00	3.269	649.5	824.1
Total / average	-	3.253	2,000.0	2,521.0

All three prices exceed marginal cost. AWS_like has the lowest assigned cost, the lowest equilibrium price and the largest demand and profit. Azure_like has the highest assigned cost and price, and the lowest demand and profit. The approximate demand shares are 38.6%, 29.0% and 32.5%, respectively; rounding explains the slight discrepancy in their sum.

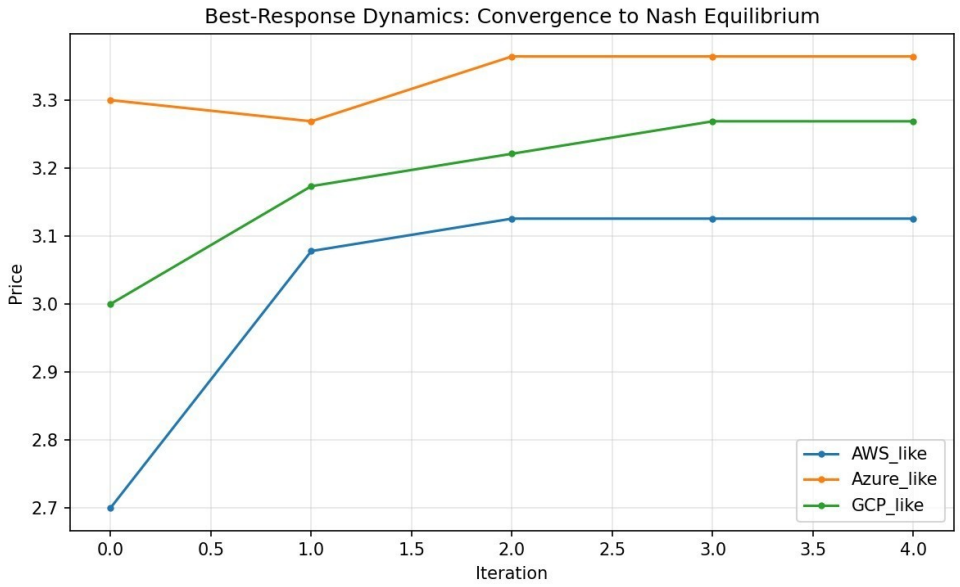


Figure 1. Best-response price trajectories for the three-provider scenario.

Starting prices are 2.70, 3.30 and 3.00. The process stabilises after four update rounds. The Python output labels the final round as iteration 3 because its counter starts at zero.

5. Results: Competition Experiment

The second experiment compares two to six symmetric providers. Every provider has marginal cost 2.0, and α is fixed at 1.5. Unlike the first experiment, base demand is set to $D_0 = 1{,}000 \times N / 3$, where N is the provider count.

Table 2. Outcomes as the number of symmetric providers increases.

Providers	Average price	Total demand	Average profit	Total profit
2	3.317	666.7	438.9	878
3	2.982	1,000.0	327.5	982
4	2.887	1,333.3	295.6	1,183
5	2.839	1,666.7	279.7	1,399
6	2.791	2,000.0	263.8	1,583

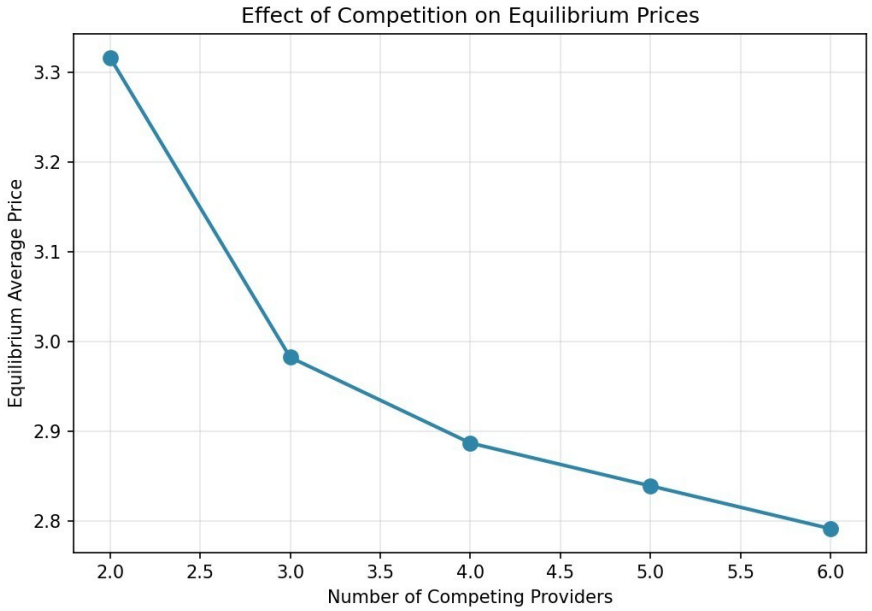


Figure 2. Average equilibrium price in markets with two to six providers.

Average price falls from 3.317 to 2.791, while average profit per provider falls from 438.9 to 263.8. Under the selected assumptions, additional competitors reduce the equilibrium markup.

Total demand rises by construction, and the reported rise in total profit must be interpreted alongside that change. The experiment does not establish that real market demand increases when a provider enters. The reduction in price is the more direct competition result.

6. Discussion, Limitations and Conclusion

6.1 Interpretation of Provider Outcomes

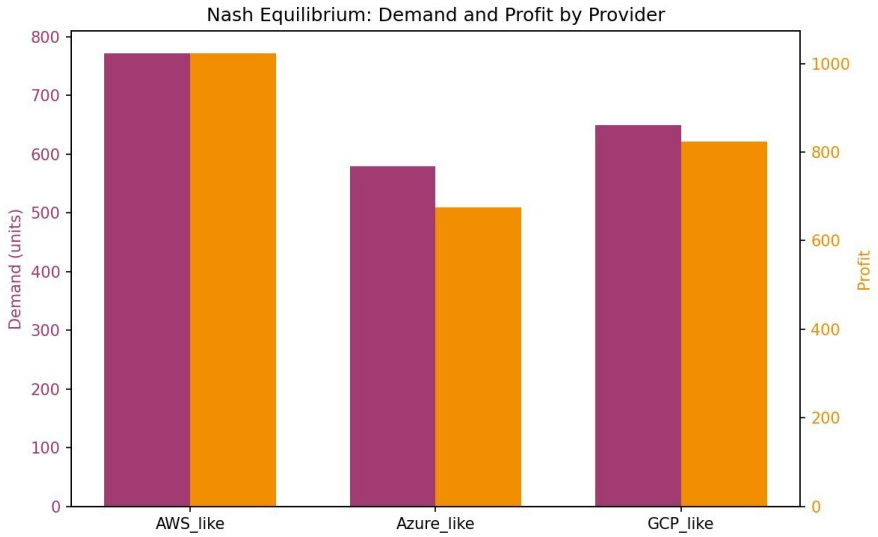


Figure 3. Demand and profit by provider; the chart uses separate vertical scales.

The three-provider example illustrates how cost differences affect competitive outcomes. The lowest-cost provider can charge less while retaining a larger margin. These results follow from the assigned parameters and should not be interpreted as a ranking of actual cloud providers.

6.2 Limitations and Future Work

Numerical approximation. Results depend on the finite price grid and its bounds. Best-response convergence in the tested scenarios is not a general convergence guarantee.

Demand assumptions. Each scenario allocates all configured demand among the providers. An outside option would allow customers to decline a purchase.

Operating assumptions. Capacity limits, fixed costs, service quality, latency, reliability, switching costs and customer segments are excluded.

Calibration. Parameters are illustrative. Future work should use a specified workload, billing period and market data, and investigate capacity constraints and time-varying pricing.

6.3 Conclusion

The project provides an executable teaching model of strategic pricing. Best-response prices stabilise in the reported scenarios; the lowest-cost provider earns the largest profit in the heterogeneous example; and increasing the number of symmetric providers lowers average equilibrium price. The model is useful for classroom exploration, provided its demand assumptions and numerical limits are retained when interpreting the results.

References and Reproducibility

References

[1] J. Nash, “Non-Cooperative Games,” *Annals of Mathematics*, vol. 54, no. 2, pp. 286-295, 1951.

[2] D. McFadden, “Conditional Logit Analysis of Qualitative Choice Behavior,” in *Frontiers in Econometrics*, P. Zarembka, Ed. New York: Academic Press, 1974.

[3] D. Fudenberg and J. Tirole, *Game Theory*. Cambridge, MA: MIT Press, 1991.

[4] K. E. Train, *Discrete Choice Methods with Simulation*, 2nd ed. Cambridge: Cambridge University Press, 2009.

Project Access

Source repository:
https://github.com/therishisenpai/gametheoryproject

Interactive dashboard:
https://therishisenpai.github.io/gametheoryproject/

Reproducing the Python Analysis

From the project directory, install the dependencies and run the analysis:

```
pip install -r requirements.txt
python cloud_pricing_game.py
```

The script prints the equilibrium and competition tables and saves convergence.png, competition_effect.png and demand_profit.png in the outputs directory. The first experiment uses demand 2,000 and $\alpha = 1.2$; the competition comparison uses $\alpha = 1.5$ and demand $1{,}000 \times N / 3$.

Running the Browser Demonstration

Open index.html with styles.css and app.js in the same directory, or use the hosted dashboard. No backend, API key or third-party JavaScript library is required. For the optional numerical smoke test, use Node.js to run:

```
node tests/model_smoke_test.js
```

The optional test checks selected baseline and competition outputs. It is separate from the browser application and does not require a cloud account.

Appendix A. Dashboard Screenshots

The following screenshots document the supplied browser interface and default results. The numerical tables in Sections 4 and 5 provide the results in a format suited to printing.

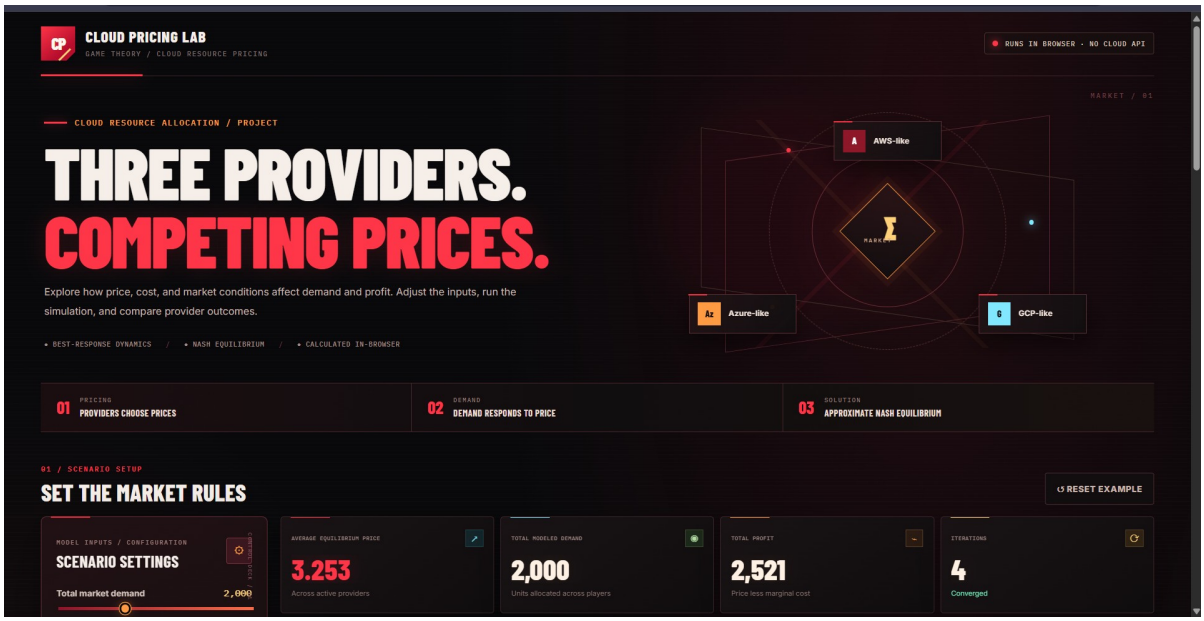


Figure A1. Dashboard landing view and summary indicators.

The summary reports average price 3.253, allocated demand 2,000, total profit approximately 2,521, and four update rounds.

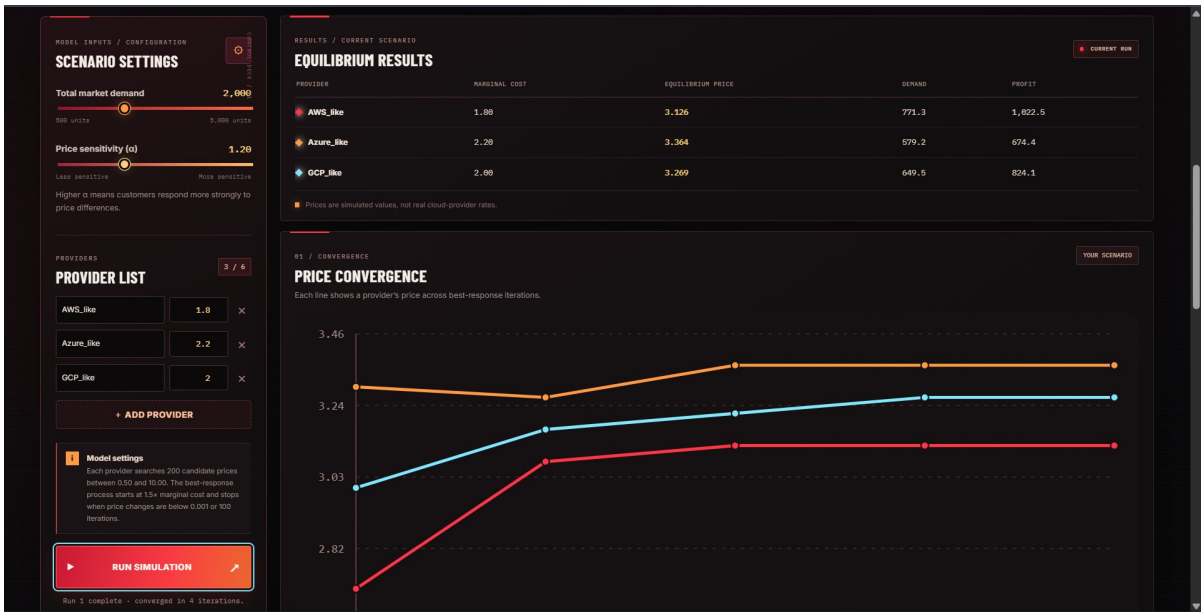


Figure A2. Scenario controls, equilibrium results and price convergence.

Default controls show demand 2,000, α = 1.20 and marginal costs 1.8, 2.2 and 2.0.

Appendix A. Dashboard Screenshots (continued)



Figure A3. Competition comparison and provider demand and profit.

The competition panel reports the separate symmetric-provider experiment, using $\alpha = 1.5$ and demand scaled with provider count.

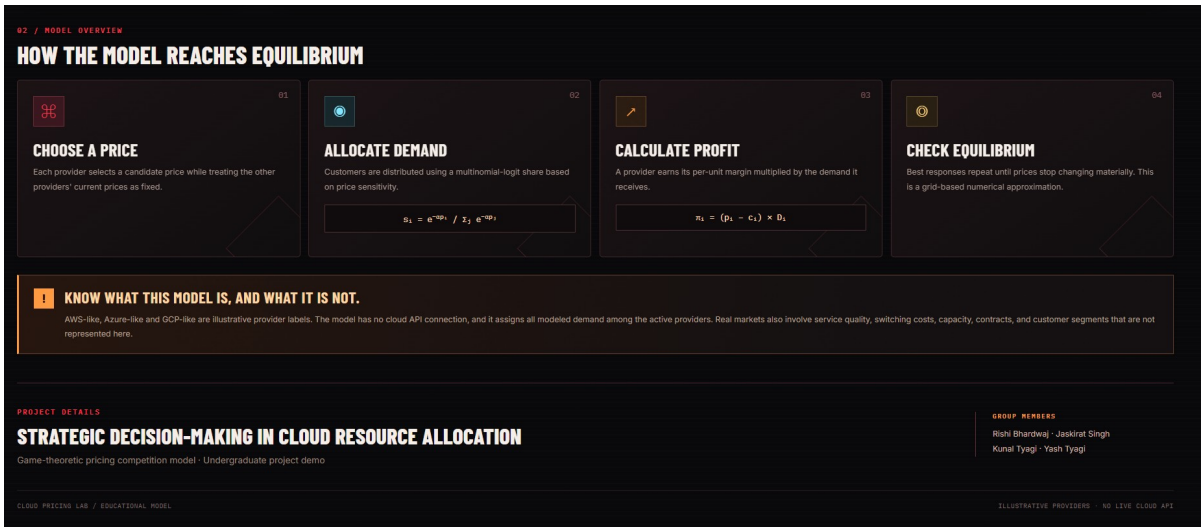


Figure A4. Model explanation and project details.

The interface describes price choice, demand allocation, profit calculation and equilibrium checking, and identifies the illustrative scope of the model.